7

Arrays and Vectors

OBJECTIVES

- To use the array data structure to represent a set of related data items.
- To use arrays to store, sort and search lists and tables of values.
- To declare arrays, initialize arrays and refer to the individual elements of arrays.

OBJECTIVES

- To pass arrays to functions.
- Basic searching and sorting techniques.
- To declare and manipulate multidimensional arrays.
- To use C++ Standard Library class template `vector`.

7.2 Arrays

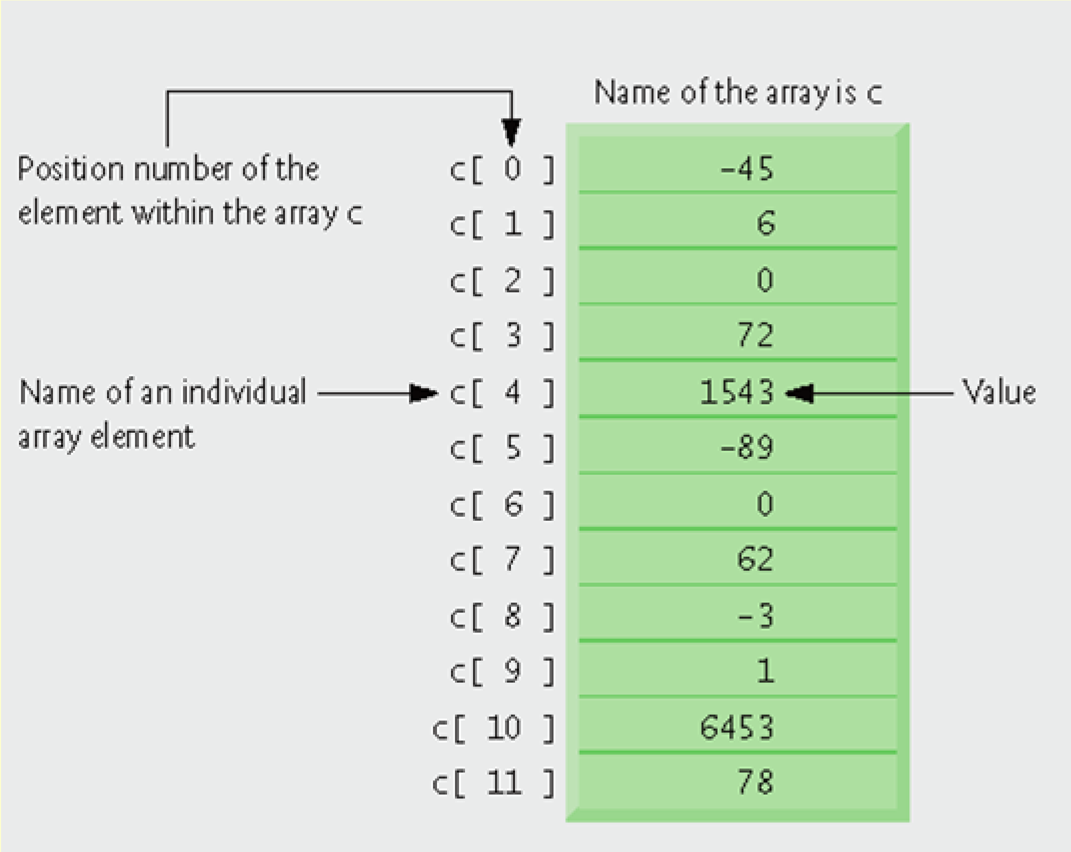
- **Array: Consecutive group of memory locations**
  - **All of which have the same type**
- **Index**
  - **Position number used to refer to a specific location/element**
  - **Also called subscript**
  - **First element has index zero**
  - **Example:**

    ```
    c[11] += 2;
    ```

    - **Adds `2` to array element `c[ 11 ]`.**

7.2 Arrays (Cont.)

- **Example**
  - **Brackets used to enclose an array subscript are actually an operator in C++.**



7.3 Declaring Arrays

- **Declaring an array**
  - **Arrays occupy space in memory.**
  - **Programmer specifies type and number of elements.**
  - **Example**

```
int c[ 12 ];
```

    - **`c` is an array of `12` `int`s**

7.3 Declaring Arrays

- **Array's size must be an integer constant greater than zero.**
- **Arrays can be declared to contain values of any non-reference data type.**
- **Multiple arrays of the same type can be declared in a single declaration.**
  - **Use a comma-separated list of names and sizes.**

7.4 Examples Using Arrays

- Using a loop to initialize the array's elements
  - Declare array, specify number of elements.
  - Use repetition statement to loop for each element.
    - Use body of repetition statement to initialize each individual array element.

```cpp
// Fig. 7.3: fig07_03.cpp
// Initializing an array.
#include <iostream>
using std::cout;
using std::endl;

#include <iomanip>
using std::setw;

int main()
{
   int n[ 10 ]; // n is an array of 10 integers

   // initialize elements of array n to 0
   for ( int i = 0; i < 10; i++ )
      n[ i ] = 0; // set element at location i to 0

   cout << "Element" << setw( 13 ) << "Value" << endl;
```

Outline

fig07_03.cpp

(1 of 2)

```
   // output each array element's value
   for ( int j = 0; j < 10; j++ )
      cout << setw( 7 ) << j << setw( 13 ) << n[ j ] << endl;

   return 0; // indicates successful termination
} // end main
```

```
Element        Value
      0            0
      1            0
      2            0
      3            0
      4            0
      5            0
      6            0
      7            0
      8            0
      9            0
```

Outline

fig07_03.cpp

(2 of 2)

7.4 Examples Using Arrays (Cont.)

- **Initializing an array in a declaration with an initializer list**
  - **Example:**

    ```
    int n[] = { 10, 20, 30, 40, 50 };
    ```

  - **The compiler determines the size of the array based on the size of the initializer list.**

7.4 Examples Using Arrays (Cont.)

- **If fewer initializers than elements in the array, remaining elements are initialized to zero.**
- **Example**

```
int n[ 10 ] = { 0 };
```

- **If more initializers than elements in the array**
  - **Compilation error**

```cpp
// Fig. 7.4: fig07_04.cpp
// Initializing an array in a declaration.
#include <iostream>
using std::cout;
using std::endl;

#include <iomanip>
using std::setw;

int main()
{
   // use initializer list to initialize array n
   int n[ 10 ] = { 32, 27, 64, 18, 95, 14, 90, 70, 60, 37 };

   cout << "Element" << setw( 13 ) << "Value" << endl;
```

```
    // output each array element's value
    for ( int i = 0; i < 10; i++ )
       cout << setw( 7 ) << i << setw( 13 ) << n[ i ] << endl;

    return 0; // indicates successful termination
} // end main
```

```
Element        Value
      0           32
      1           27
      2           64
      3           18
      4           95
      5           14
      6           90
      7           70
      8           60
      9           37
```

```cpp
// Fig. 7.5: fig07_05.cpp
// Set array s to the even integers from 2 to 20.
#include <iostream>
using std::cout;
using std::endl;

#include <iomanip>
using std::setw;

int main()
{
   // constant variable can be used to specify array size
   const int arraySize = 10;

   int s[ arraySize ]; // array s has 10 elements

   for ( int i = 0; i < arraySize; i++ ) // set the values
      s[ i ] = 2 + 2 * i;
```

Outline

fig07_05.cpp

(1 of 2)

```
   cout << "Element" << setw( 13 ) << "Value" << endl;

   // output contents of array s in tabular format
   for ( int j = 0; j < arraySize; j++ )
      cout << setw( 7 ) << j << setw( 13 ) << s[ j ] << endl;

   return 0; // indicates successful termination
} // end main
```

```
Element        Value
      0            2
      1            4
      2            6
      3            8
      4           10
      5           12
      6           14
      7           16
      8           18
      9           20
```

Outline

fig07_05.cpp

(2 of 2)

7.4 Examples Using Arrays (Cont.)

- **Constant variables**
  - **Declared using the `const` qualifier**
  - **Also called name constants or read-only variables**
  - **Must be initialized with a constant expression when they are declared and cannot be modified thereafter.**
  - **Can be placed anywhere a constant expression is expected.**
  - **Using constant variables to specify array sizes makes programs more scalable and eliminates “magic numbers”.**

```cpp
// Fig. 7.6: fig07_06.cpp
// Using a properly initialized constant variable.
#include <iostream>
using std::cout;
using std::endl;

int main()
{
   const int x = 7; // initialized constant variable

   cout << "The value of constant variable x is: " << x << endl;

   return 0; // indicates successful termination
} // end main
```

```
The value of constant variable x is: 7
```


fig07_06.cpp

```cpp
// Fig. 7.7: fig07_07.cpp
// A const variable must be initialized.

int main()
{
   const int x; // Error: x must be initialized

   x = 7; // Error: cannot modify a const variable

   return 0; // indicates successful termination
} // end main
```

Borland C++ command-line compiler error message:

```
Error E2304 fig07_07.cpp 6: Constant variable 'x' must be initialized
   in function main()
Error E2024 fig07_07.cpp 8: Cannot modify a const object in function main()
```

Microsoft Visual C++.NET compiler error message:

```
C:\cpphtp5_examples\ch07\fig07_07.cpp(6) : error C2734: 'x' : const object
   must be initialized if not extern
C:\cpphtp5_examples\ch07\fig07_07.cpp(8) : error C2166: l-value specifies
   const object
```

GNU C++ compiler error message:

```
fig07_07.cpp:6: error: uninitialized const `x'
fig07_07.cpp:8: error: assignment of read-only variable `x'
```

Outline

fig07_07.cpp

(1 of 1)

7.4 Examples Using Arrays (Cont.)

- **Summing the elements of an array**
  - **Array elements can represent a series of values**
  - **We can sum these values.**
  - **Use repetition statement to loop through each element.**
    - **Add element value to a total.**

```
// Fig. 7.8: fig07_08.cpp
// Compute the sum of the elements of the array.
#include <iostream>
using std::cout;
using std::endl;

int main()
{
   const int arraySize = 10; // constant variable indicating size of array
   int a[ arraySize ] = { 87, 68, 94, 100, 83, 78, 85, 91, 76, 87 };
   int total = 0;

   // sum contents of array a
   for ( int i = 0; i < arraySize; i++ )
      total += a[ i ];

   cout << "Total of array elements: " << total << endl;

   return 0; // indicates successful termination
} // end main
```

```
Total of array elements: 849
```

Outline

fig07_08.cpp

(1 of 1)

7.4 Examples Using Arrays (Cont.)

- **Using bar charts to display array data graphically**
  - **Present data in graphical manner.**
  - **Example: bar chart**
  - **Examine the distribution of grades.**
  - **Nested `for` statement used to output bars**

```cpp
// Fig. 7.9: fig07_09.cpp
// Bar chart printing program.
#include <iostream>
using std::cout;
using std::endl;

#include <iomanip>
using std::setw;

int main()
{
   const int arraySize = 11;
   int n[ arraySize ] = { 0, 0, 0, 0, 0, 0, 1, 2, 4, 2, 1 };

   cout << "Grade distribution:" << endl;

   // for each element of array n, output a bar of the chart
   for ( int i = 0; i < arraySize; i++ )
   {
      // output bar labels ("0-9:", ..., "90-99:", "100:" )
      if ( i == 0 )
         cout << "  0-9: ";
      else if ( i == 10 )
         cout << "  100: ";
      else
         cout << i * 10 << "-" << ( i * 10 ) + 9 << ": ";
```


fig07_09.cpp

```cpp

      // print bar of asterisks
      for ( int stars = 0; stars < n[ i ]; stars++ )
         cout << '*';

      cout << endl; // start a new line of output
   } // end outer for

   return 0; // indicates successful termination
} // end main
```

```
Grade distribution:
  0-9:
10-19:
20-29:
30-39:
40-49:
50-59:
60-69: *
70-79: **
80-89: ****
90-99: **
  100: *
```

7.4 Examples Using Arrays (Cont.)

- **Using the elements of an array as counters**
  - **Use a series of counter variables to summarize data.**
  - **Counter variables make up an array.**
  - **Store frequency values.**

```cpp
// Fig. 7.10: fig07_10.cpp
// Roll a six-sided die 6,000,000 times.
#include <iostream>
using std::cout;
using std::endl;

#include <iomanip>
using std::setw;

#include <cstdlib>
using std::rand;
using std::srand;

#include <ctime>
using std::time;

int main()
{
   const int arraySize = 7; // ignore element zero
   int frequency[ arraySize ] = { 0 };

   srand( time( 0 ) ); // seed random number generator

   // roll die 6,000,000 times; use die value as frequency index
   for ( int roll = 1; roll <= 6000000; roll++ )
      frequency[ 1 + rand() % 6 ]++;
```

Outline

fig07_10.cpp

(1 of 2)

```
   cout << "Face" << setw( 13 ) << "Frequency" << endl;

   // output each array element's value
   for ( int face = 1; face < arraySize; face++ )
      cout << setw( 4 ) << face << setw( 13 ) << frequency[ face ]
         << endl;

   return 0; // indicates successful termination
} // end main
```

```
Face    Frequency
   1      1000167
   2      1000149
   3      1000152
   4       998748
   5       999626
   6      1001158
```

Outline

fig07_10.cpp

(2 of 2)

7.4 Examples Using Arrays (Cont.)

- **Using arrays to summarize survey results**
  - **40 students rate the quality of food.**
    - **1-10 rating scale: 1 means awful, 10 means excellent**
  - **Place 40 responses in an array of integers.**
  - **Summarize results.**
  - **Each element of the array used as a counter for one of the survey responses.**

```cpp
// Fig. 7.11: fig07_11.cpp
// Student poll program.
#include <iostream>
using std::cout;
using std::endl;

#include <iomanip>
using std::setw;

int main()
{
   // define array sizes
   const int responseSize = 40; // size of array responses
   const int frequencySize = 11; // size of array frequency

   // place survey responses in array responses
   const int responses[ responseSize ] = { 1, 2, 6, 4, 8, 5, 9, 7, 8,
      10, 1, 6, 3, 8, 6, 10, 3, 8, 2, 7, 6, 5, 7, 6, 8, 6, 7,
      5, 6, 6, 5, 6, 7, 5, 6, 4, 8, 6, 8, 10 };

   // initialize frequency counters to 0
   int frequency[ frequencySize ] = { 0 };

   // for each answer, select responses element and use that value
   // as frequency subscript to determine element to increment
   for ( int answer = 0; answer < responseSize; answer++ )
      frequency[ responses[ answer ] ]++;

   cout << "Rating" << setw( 17 ) << "Frequency" << endl;
```

Outline

fig07_11.cpp

(1 of 2)

```cpp

   // output each array element's value
   for ( int rating = 1; rating < frequencySize; rating++ )
      cout << setw( 6 ) << rating << setw( 17 ) << frequency[ rating ]
         << endl;

   return 0; // indicates successful termination
} // end main
```

```
Rating        Frequency
     1                2
     2                2
     3                2
     4                2
     5                5
     6               11
     7                5
     8                7
     9                1
    10                3
```


fig07_11.cpp

7.4 Examples Using Arrays (Cont.)

- **Using character arrays to store and manipulate strings**
  - **Arrays may be of any type, including `char`s.**
  - **We can store character strings in `char` arrays.**
  - **Can be initialized using a string literal.**
  - **Example**

```
char string1[] = "Hi";
```

equivalent to

```
char string1[] = { 'H', 'i', '\0' };
```

7.4 Examples Using Arrays (Cont.)

- **Array contains each character plus a special string-termination character called the null character (`'\0'`).**
- **Can also be initialized with individual character constants in an initializer list.**

```
char string1[] =
      { 'f', 'i', 'r', 's', 't', '\0' };
```

- **Can also input a string directly into a character array from the keyboard using `cin` and `>>`.**
- **A character array representing a null-terminated string can be output with `cout` and `<<`.**

```cpp
// Fig. 7.12: fig07_12.cpp
// Treating character arrays as strings.
#include <iostream>
using std::cout;
using std::cin;
using std::endl;

int main()
{
   char string1[ 20 ]; // reserves 20 characters
   char string2[] = "string literal"; // reserves 15 characters

   // read string from user into array string1
   cout << "Enter the string \"hello there\": ";
   cin >> string1; // reads "hello" [space terminates input]

   // output strings
   cout << "string1 is: " << string1 << "\nstring2 is: " << string2;

   cout << "\nstring1 with spaces between characters is:\n";

```


fig07_12.cpp

(1 of 2)

```cpp
   // output characters until null character is reached
   for ( int i = 0; string1[ i ] != '\0'; i++ )
      cout << string1[ i ] << ' ';

   cin >> string1; // reads "there"
   cout << "\nstring1 is: " << string1 << endl;

   return 0; // indicates successful termination
} // end main
```

```
Enter the string "hello there": hello there
string1 is: hello
string2 is: string literal
string1 with spaces between characters is:
h e l l o
string1 is: there
```

Outline

fig07_12.cpp

(2 of 2)

7.4 Examples Using Arrays (Cont.)

- **`static` local arrays and automatic local arrays**
  - **A `static` local variable in a function**
    - **Exists for the duration of the program.**
    - **But is visible only in the function body.**
  - **A `static` local array**
    - **Exists for the duration of the program.**
    - **Is initialized when its declaration is first encountered.**
      - **All elements are initialized to zero if not explicitly initialized.**
        - **This does not happen for automatic local arrays.**

```cpp
// Fig. 7.13: fig07_13.cpp
// Static arrays are initialized to zero.
#include <iostream>
using std::cout;
using std::endl;

void staticArrayInit( void ); // function prototype
void automaticArrayInit( void ); // function prototype

int main()
{
   cout << "First call to each function:\n";
   staticArrayInit();
   automaticArrayInit();

   cout << "\n\nSecond call to each function:\n";
   staticArrayInit();
   automaticArrayInit();
   cout << endl;

   return 0; // indicates successful termination
} // end main
```

Outline

fig07_13.cpp

(1 of 3)

```cpp

// function to demonstrate a static local array
void staticArrayInit( void )
{
   // initializes elements to 0 first time function is called
   static int array1[ 3 ]; // static local array

   cout << "\nValues on entering staticArrayInit:\n";

   // output contents of array1
   for ( int i = 0; i < 3; i++ )
      cout << "array1[" << i << "] = " << array1[ i ] << "  ";

   cout << "\nValues on exiting staticArrayInit:\n";

   // modify and output contents of array1
   for ( int j = 0; j < 3; j++ )
      cout << "array1[" << j << "] = " << ( array1[ j ] += 5 ) << "  ";
} // end function staticArrayInit

// function to demonstrate an automatic local array
void automaticArrayInit( void )
{
   // initializes elements each time function is called
   int array2[ 3 ] = { 1, 2, 3 }; // automatic local array

   cout << "\n\nValues on entering automaticArrayInit:\n";
```

```cpp

   // output contents of array2
   for ( int i = 0; i < 3; i++ )
      cout << "array2[" << i << "] = " << array2[ i ] << "  ";

   cout << "\nValues on exiting automaticArrayInit:\n";

   // modify and output contents of array2
   for ( int j = 0; j < 3; j++ )
      cout << "array2[" << j << "] = " << ( array2[ j ] += 5 ) << "  ";
} // end function automaticArrayInit
```

```
First call to each function:

Values on entering staticArrayInit:
array1[0] = 0  array1[1] = 0  array1[2] = 0
Values on exiting staticArrayInit:
array1[0] = 5  array1[1] = 5  array1[2] = 5

Values on entering automaticArrayInit:
array2[0] = 1  array2[1] = 2  array2[2] = 3
Values on exiting automaticArrayInit:
array2[0] = 6  array2[1] = 7  array2[2] = 8

Second call to each function:

Values on entering staticArrayInit:
array1[0] = 5  array1[1] = 5  array1[2] = 5
Values on exiting staticArrayInit:
array1[0] = 10  array1[1] = 10  array1[2] = 10

Values on entering automaticArrayInit:
array2[0] = 1  array2[1] = 2  array2[2] = 3
Values on exiting automaticArrayInit:
array2[0] = 6  array2[1] = 7  array2[2] = 8
```

Class Work

- **Declare an array with three elements that have the initial values of 2, 3, and 4.**

Answer:

Class Work

- **Write down all the elements of the array below:**

```
char name[] = “John”;
```

Answer:

```
name[0] = ‘J’
```

7.5 Passing Arrays to Functions

- **To pass an array argument to a function**
  - **Specify array name without brackets.**
    - **Array `hourlyTemperatures` is declared as**

      ```
      int hourlyTemperatures[ 24 ];
      ```

    - **The function call**

      ```
      modifyArray( hourlyTemperatures, 24 );
      ```

      passes array `hourlyTemperatures` and its size to function `modifyArray`
  - **Array size is normally passed as another argument so the function can process the specific number of elements in the array.**

7.5 Passing Arrays to Functions (Cont.)

- **Arrays are passed by reference.**
  - **Function call actually passes starting address of array.**
- **Individual array elements passed by value**
- **Functions that take arrays as arguments**
  - **Function parameter list must specify array parameter**
    - **Example**
      - `void modArray( int b[], int arraySize );`

```cpp
// Fig. 7.14: fig07_14.cpp
// Passing arrays and individual array elements to functions.
#include <iostream>
using std::cout;
using std::endl;

#include <iomanip>
using std::setw;

void modifyArray( int [], int ); // appears strange
void modifyElement( int );

int main()
{
   const int arraySize = 5; // size of array a
   int a[ arraySize ] = { 0, 1, 2, 3, 4 }; // initialize array a

   cout << "Effects of passing entire array by reference:"
      << "\n\nThe values of the original array are:\n";

   // output original array elements
   for ( int i = 0; i < arraySize; i++ )
      cout << setw( 3 ) << a[ i ];

   cout << endl;

   // pass array a to modifyArray by reference
   modifyArray( a, arraySize );
   cout << "The values of the modified array are:\n";
```

Outline

fig07_14.cpp

(1 of 3)

```
   // output modified array elements
   for ( int j = 0; j < arraySize; j++ )
      cout << setw( 3 ) << a[ j ];

   cout << "\n\n\nEffects of passing array element by value:"
      << "\n\na[3] before modifyElement: " << a[ 3 ] << endl;

   modifyElement( a[ 3 ] ); // pass array element a[ 3 ] by value
   cout << "a[3] after modifyElement: " << a[ 3 ] << endl;

   return 0; // indicates successful termination
} // end main

// in function modifyArray, "b" points to the original array "a" in memory
void modifyArray( int b[], int sizeOfArray )
{
   // multiply each array element by 2
   for ( int k = 0; k < sizeOfArray; k++ )
      b[ k ] *= 2;
} // end function modifyArray
```

Outline

fig07_14.cpp

(2 of 3)

```cpp

// in function modifyElement, "e" is a local copy of
// array element a[ 3 ] passed from main
void modifyElement( int e )
{
   // multiply parameter by 2
   cout << "Value of element in modifyElement: " << ( e *= 2 ) << endl;
} // end function modifyElement
```

```
Effects of passing entire array by reference:

The values of the original array are:
  0  1  2  3  4
The values of the modified array are:
  0  2  4  6  8


Effects of passing array element by value:

a[3] before modifyElement: 6
Value of element in modifyElement: 12
a[3] after modifyElement: 6
```

Outline

fig07_14.cpp

(3 of 3)

7.5 Passing Arrays to Functions (Cont.)

- `const` **array parameters**
  - **Qualifier** `const`
  - **Elements in the array are constant in the function body.**
  - **Enables programmer to prevent accidental modification of data.**

```cpp
// Fig. 7.15: fig07_15.cpp
// Demonstrating the const type qualifier.
#include <iostream>
using std::cout;
using std::endl;

void tryToModifyArray( const int [] ); // function prototype

int main()
{
   int a[] = { 10, 20, 30 };

   tryToModifyArray( a );
   cout << a[ 0 ] << ' ' << a[ 1 ] << ' ' << a[ 2 ] << '\n';

   return 0; // indicates successful termination
} // end main

```

Outline

fig07_15.cpp

(1 of 2)

```
// In function tryToModifyArray, "b" cannot be used
// to modify the original array "a" in main.
void tryToModifyArray( const int b[] )
{
   b[ 0 ] /= 2; // error
   b[ 1 ] /= 2; // error
   b[ 2 ] /= 2; // error
} // end function tryToModifyArray
```

Borland C++ command-line compiler error message:

```
Error E2024 fig07_15.cpp 23: Cannot modify a const object
   in function tryToModifyArray(const int * const)
Error E2024 fig07_15.cpp 24: Cannot modify a const object
   in function tryToModifyArray(const int * const)
Error E2024 fig07_15.cpp 25: Cannot modify a const object
   in function tryToModifyArray(const int * const)
```

Microsoft Visual C++.NET compiler error message:

```
C:\cpphtp5_examples\ch07\fig07_15.cpp(23) : error C2166: l-value specifies
   const object
C:\cpphtp5_examples\ch07\fig07_15.cpp(24) : error C2166: l-value specifies
   const object
C:\cpphtp5_examples\ch07\fig07_15.cpp(25) : error C2166: l-value specifies
   const object
```

GNU C++ compiler error message:

```
fig07_15.cpp:23: error: assignment of read-only location
fig07_15.cpp:24: error: assignment of read-only location
fig07_15.cpp:25: error: assignment of read-only location
```

7.6 Case Study: Class `GradeBook` Using an Array to Store Grades

- **Class `GradeBook`**
  - **Represent a grade book that stores and analyzes grades**
  - **Can now store grades in an array**
- **`static` data members**
  - **Also called class variables**
  - **Variables for which each object of a class does not have a separate copy**
    - **One copy is shared among all objects of the class**
  - **Can be accessed even when no objects of the class exist**
    - **Use the class name followed by the binary scope resolution operator and the name of the `static` data member**

```cpp
// Fig. 7.16: GradeBook.h
// Definition of class GradeBook that uses an array to store test grades.
// Member functions are defined in GradeBook.cpp

#include <string> // program uses C++ Standard Library string class
using std::string;

// GradeBook class definition
class GradeBook
{
public:
   // constant -- number of students who took the test
   const static int students = 10; // note public data

   // constructor initializes course name and array of grades
   GradeBook( string, const int [] );

   void setCourseName( string ); // function to set the course name
   string getCourseName(); // function to retrieve the course name
   void displayMessage(); // display a welcome message
   void processGrades(); // perform various operations on the grade data
   int getMinimum(); // find the minimum grade for the test
   int getMaximum(); // find the maximum grade for the test
   double getAverage(); // determine the average grade for the test
   void outputBarChart(); // output bar chart of grade distribution
   void outputGrades(); // output the contents of the grades array
private:
   string courseName; // course name for this grade book
   int grades[ students ]; // array of student grades
}; // end class GradeBook
```


fig07_16.cpp

(1 of 1)

```
// Fig. 7.17: GradeBook.cpp
// Member-function definitions for class GradeBook that
// uses an array to store test grades.
#include <iostream>
using std::cout;
using std::cin;
using std::endl;
using std::fixed;

#include <iomanip>
using std::setprecision;
using std::setw;

#include "GradeBook.h" // GradeBook class definition

// constructor initializes courseName and grades array
GradeBook::GradeBook( string name, const int gradesArray[] )
{
   setCourseName( name ); // initialize courseName

   // copy grades from gradeArray to grades data member
   for ( int grade = 0; grade < students; grade++ )
      grades[ grade ] = gradesArray[ grade ];
} // end GradeBook constructor

// function to set the course name
void GradeBook::setCourseName( string name )
{
   courseName = name; // store the course name
} // end function setCourseName
```

Outline

fig07_17.cpp

(1 of 6)

```
// function to retrieve the course name
string GradeBook::getCourseName()
{
   return courseName;
} // end function getCourseName

// display a welcome message to the GradeBook user
void GradeBook::displayMessage()
{
   // this statement calls getCourseName to get the
   // name of the course this GradeBook represents
   cout << "Welcome to the grade book for\n" << getCourseName() << "!"
      << endl;
} // end function displayMessage

// perform various operations on the data
void GradeBook::processGrades()
{
   // output grades array
   outputGrades();

   // call function getAverage to calculate the average grade
   cout << "\nClass average is " << setprecision( 2 ) << fixed <<
      getAverage() << endl;

   // call functions getMinimum and getMaximum
   cout << "Lowest grade is " << getMinimum() << "\nHighest grade is "
      << getMaximum() << endl;
```

Outline

fig07_17.cpp

(2 of 6)

```
   // call function outputBarChart to print grade distribution chart
   outputBarChart();
} // end function processGrades

// find minimum grade
int GradeBook::getMinimum()
{
   int lowGrade = 100; // assume lowest grade is 100

   // loop through grades array
   for ( int grade = 0; grade < students; grade++ )
   {
      // if current grade lower than lowGrade, assign it to lowGrade
      if ( grades[ grade ] < lowGrade )
         lowGrade = grades[ grade ]; // new lowest grade
   } // end for

   return lowGrade; // return lowest grade
} // end function getMinimum

// find maximum grade

int GradeBook::getMaximum()
```

```
{
   int highGrade = 0; // assume highest grade is 0

   // loop through grades array
   for ( int grade = 0; grade < students; grade++ )
   {
      // if current grade higher than highGrade, assign it to highGrade
      if ( grades[ grade ] > highGrade )
         highGrade = grades[ grade ]; // new highest grade
   } // end for

   return highGrade; // return highest grade
} // end function getMaximum

// determine average grade for test
double GradeBook::getAverage()
{
   int total = 0; // initialize total

   // sum grades in array
   for ( int grade = 0; grade < students; grade++ )
      total += grades[ grade ];

   // return average of grades
   return static_cast< double >( total ) / students;
} // end function getAverage

// output bar chart displaying grade distribution
void GradeBook::outputBarChart()
```

Outline

fig07_17.cpp

(4 of 6)

```
{
   cout << "\nGrade distribution:" << endl;

   // stores frequency of grades in each range of 10 grades
   const int frequencySize = 11;
   int frequency[ frequencySize ] = { 0 };

   // for each grade, increment the appropriate frequency
   for ( int grade = 0; grade < students; grade++ )
      frequency[ grades[ grade ] / 10 ]++;

   // for each grade frequency, print bar in chart
   for ( int count = 0; count < frequencySize; count++ )
   {
      // output bar labels ("0-9:", ..., "90-99:", "100:" )
      if ( count == 0 )
         cout << "  0-9: ";
      else if ( count == 10 )
         cout << "  100: ";
      else
         cout << count * 10 << "-" << ( count * 10 ) + 9 << ": ";

      // print bar of asterisks
      for ( int stars = 0; stars < frequency[ count ]; stars++ )
         cout << '*';

      cout << endl; // start a new line of output
```

Outline

fig07_17.cpp

(5 of 6)

```
   } // end outer for
} // end function outputBarChart

// output the contents of the grades array
void GradeBook::outputGrades()
{
   cout << "\nThe grades are:\n\n";

   // output each student's grade
   for ( int student = 0; student < students; student++ )
      cout << "Student " << setw( 2 ) << student + 1 << ": " << setw( 3 )
         << grades[ student ] << endl;
} // end function outputGrades
```

Outline

fig07_17.cpp

(6 of 6)

Outline

```
// Fig. 7.18: fig07_18.cpp
// Creates GradeBook object using an array of grades.

#include "GradeBook.h" // GradeBook class definition

// function main begins program execution
int main()
{
   // array of student grades
   int gradesArray[ GradeBook::students ] =
      { 87, 68, 94, 100, 83, 78, 85, 91, 76, 87 };

   GradeBook myGradeBook(
      "CS101 Introduction to C++ Programming", gradesArray );
   myGradeBook.displayMessage();
   myGradeBook.processGrades();
   return 0;
} // end main
```

```
Welcome to the grade book for
CS101 Introduction to C++ Programming!

The grades are:

Student  1:  87
Student  2:  68
Student  3:  94
Student  4: 100
Student  5:  83
Student  6:  78
Student  7:  85
Student  8:  91
Student  9:  76
Student 10:  87

Class average is 84.90
Lowest grade is 68
Highest grade is 100

Grade distribution:
  0-9:
10-19:
20-29:
30-39:
40-49:
50-59:
60-69: *
70-79: **
80-89: ****
90-99: **
  100: *
```

Outline

fig07_18.cpp

(2 of 2)

7.7 Searching Arrays with Linear Search

- **Searching is required since arrays may store large amounts of data.**
- **Linear search**
  - **Compares each element of an array with a search key.**
  - **On average, a program must compare the search key with half the elements of the array.**
  - **Works well for small or unsorted arrays.**

```cpp
// Fig. 7.19: fig07_19.cpp
// Linear search of an array.
#include <iostream>
using std::cout;
using std::cin;
using std::endl;

int linearSearch( const int [], int, int ); // prototype

int main()
{
   const int arraySize = 100; // size of array a
   int a[ arraySize ]; // create array a
   int searchKey; // value to locate in array a

   for ( int i = 0; i < arraySize; i++ )
      a[ i ] = 2 * i; // create some data

   cout << "Enter integer search key: ";
   cin >> searchKey;

   // attempt to locate searchKey in array a
   int element = linearSearch( a, searchKey, arraySize );

```

Outline

fig07_19.cpp

(1 of 2)

```cpp
   // display results
   if ( element != -1 )
      cout << "Found value in element " << element << endl;
   else
      cout << "Value not found" << endl;

   return 0; // indicates successful termination
} // end main

// compare key to every element of array until location is
// found or until end of array is reached; return subscript of
// element if key or -1 if key not found
int linearSearch( const int array[], int key, int sizeOfArray )
{
   for ( int j = 0; j < sizeOfArray; j++ )
      if ( array[ j ] == key ) // if found,
         return j; // return location of key

   return -1; // key not found
} // end function linearSearch
```

```
Enter integer search key: 36
Found value in element 18
```

```
Enter integer search key: 37
Value not found
```

Outline

fig07_19.cpp

(2 of 2)

7.8 Sorting Arrays with Insertion Sort

- **Sorting data is one of the most important computing applications**
- **Insertion sort**
  - **First iteration takes second element.**
    - **If it is less than the first element, swap it with first element.**
  - **Second iteration looks at the third element.**
    - **Insert it into the correct position with respect to first two elements.**
  - **…**
  - **Simple but inefficient.**

```
// Fig. 7.20: fig07_20.cpp
// This program sorts an array's values into ascending order.
#include <iostream>
using std::cout;
using std::endl;

#include <iomanip>
using std::setw;

int main()
{
   const int arraySize = 10; // size of array a
   int data[ arraySize ] = { 34, 56, 4, 10, 77, 51, 93, 30, 5, 52 };
   int insert; // temporary variable to hold element to insert

   cout << "Unsorted array:\n";

   // output original array
   for ( int i = 0; i < arraySize; i++ )
      cout << setw( 4 ) << data[ i ];

   // insertion sort
   // loop over the elements of the array
   for ( int next = 1; next < arraySize; next++ )
   {
      insert = data[ next ]; // store the value in the current element

      int moveItem = next; // initialize location to place element
```

Outline

fig07_20.cpp

```
   // search for the location in which to put the current element
   while ( ( moveItem > 0 ) && ( data[ moveItem - 1 ] > insert ) )
   {
      // shift element one slot to the right
      data[ moveItem ] = data[ moveItem - 1 ];
      moveItem--;
   } // end while

   data[ moveItem ] = insert; // place inserted element into the array
} // end for

cout << "\nSorted array:\n";

// output sorted array
for ( int i = 0; i < arraySize; i++ )
   cout << setw( 4 ) << data[ i ];

cout << endl;
return 0; // indicates successful termination
} // end main
```

```
Unsorted array:
  34  56   4  10  77  51  93  30   5  52
Sorted array:
   4   5  10  30  34  51  52  56  77  93
```

Outline

fig07_20.cpp

(2 of 2)

7.9 Multidimensional Array

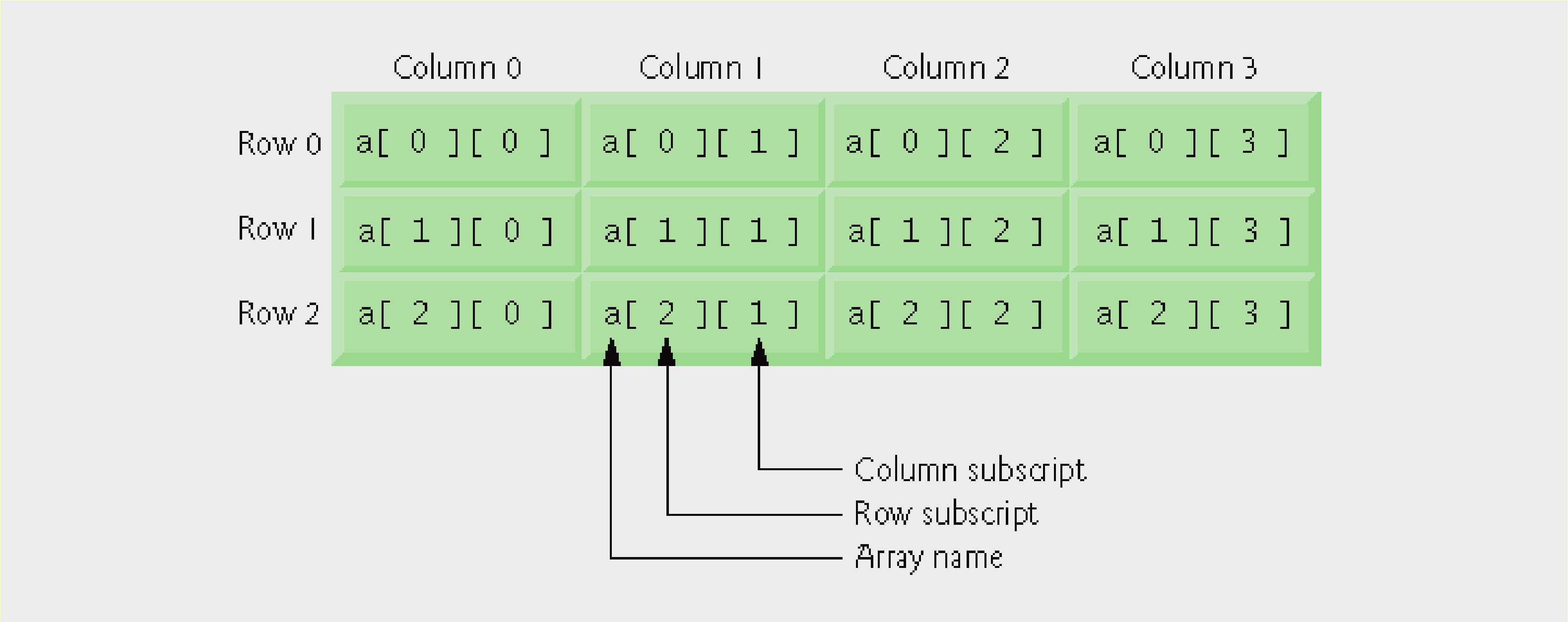
- **Multidimensional arrays with two dimensions**
  - **Called two dimensional or 2-D arrays**
  - **Represent tables of values with rows and columns**
  - **Elements referenced with two subscripts (`[ x ][ y ]`)**
  - **In general, an array with *m* rows and *n* columns is called an *m*-by-*n* array.**
- **Multidimensional arrays can have more than two dimensions.**

7.9 Multidimensional Array (Cont.)

- **Declaring and initializing two-dimensional arrays**
  - **Declaring two-dimensional array `b`**
    - `int b[ 2 ][ 2 ] = { { 1, 2 }, { 3, 4 } };`
      - **`1` and `2` initialize `b[ 0 ][ 0 ]` and `b[ 0 ][ 1 ]`**
      - **`3` and `4` initialize `b[ 1 ][ 0 ]` and `b[ 1 ][ 1 ]`**
    - `int b[ 2 ][ 2 ] = { { 1 }, { 3, 4 } };`
      - **Row `0` contains values `1` and `0` (implicitly initialized to zero).**
      - **Row `1` contains values `3` and `4`.**

7.9 Multidimensional Array (Cont.)

- **Declaring and initializing two-dimensional arrays (diagram)**



```cpp
// Fig. 7.22: fig07_22.cpp
// Initializing multidimensional arrays.
#include <iostream>
using std::cout;
using std::endl;

void printArray( const int [][ 3 ] ); // prototype

int main()
{
   int array1[ 2 ][ 3 ] = { { 1, 2, 3 }, { 4, 5, 6 } };
   int array2[ 2 ][ 3 ] = { 1, 2, 3, 4, 5 };
   int array3[ 2 ][ 3 ] = { { 1, 2 }, { 4 } };

   cout << "Values in array1 by row are:" << endl;
   printArray( array1 );

   cout << "\nValues in array2 by row are:" << endl;
   printArray( array2 );

   cout << "\nValues in array3 by row are:" << endl;
   printArray( array3 );
   return 0; // indicates successful termination
} // end main
```

```
// output array with two rows and three columns
void printArray( const int a[][ 3 ] )
{
   // loop through array's rows
   for ( int i = 0; i < 2; i++ )
   {
      // loop through columns of current row
      for ( int j = 0; j < 3; j++ )
         cout << a[ i ][ j ] << ' ';

      cout << endl; // start new line of output
   } // end outer for
} // end function printArray
```

```
Values in array1 by row are:
1 2 3
4 5 6

Values in array2 by row are:
1 2 3
4 5 0

Values in array3 by row are:
1 2 0
4 0 0
```

Outline

fig07_22.cpp

(2 of 2)

7.9 Multidimensional Array (Cont.)

- **Multidimensional array parameters**
  - **Size of first dimension is not required.**
    - **As with a one-dimensional array**
  - **Size of subsequent dimensions are required.**
    - **Compiler must know how many elements to skip to move to the second element in the first dimension.**
  - **Example**
    - `void printArray( const int a[][ 3 ] );`
      - **Function will skip row `0`'s `3` elements to access row `1`'s elements (`a[ 1 ][ x ]`).**

Class Work

- **Write down the index of each element of the following array:**

int table[3][3] = {{1, 8}, {2, 4, 6}, {5}};

Answer:

```
table[   ][   ] = 1
table[   ][   ] = 8
table[   ][   ] = 2
table[   ][   ] = 4
table[   ][   ] = 6
table[   ][   ] = 5
```

7.10 Case Study: Class `GradeBook` Using a Two-Dimensional Array

- **Class `GradeBook`**
  - **One-dimensional array**
    - **Store student grades on a single exam.**
  - **Two-dimensional array**
    - **Store multiple grades for a single student and multiple students for the class as a whole.**
      - **Each row represents a student's grades.**
      - **Each column represents all the grades the students earned for one particular exam.**

```
// Fig. 7.23: GradeBook.h
// Definition of class GradeBook that uses a
// two-dimensional array to store test grades.
// Member functions are defined in GradeBook.cpp
#include <string> // program uses C++ Standard Library string class
using std::string;

// GradeBook class definition
class GradeBook
{
public:
   // constants
   const static int students = 10; // number of students
   const static int tests = 3; // number of tests

   // constructor initializes course name and array of grades
   GradeBook( string, const int [][ tests ] );
```

Outline

fig07_23.cpp

(1 of 2)

```
   void setCourseName( string ); // function to set the course name
   string getCourseName(); // function to retrieve the course name
   void displayMessage(); // display a welcome message
   void processGrades(); // perform various operations on the grade data
   int getMinimum(); // find the minimum grade in the grade book
   int getMaximum(); // find the maximum grade in the grade book
   double getAverage( const int [], const int ); // find average of grades
   void outputBarChart(); // output bar chart of grade distribution
   void outputGrades(); // output the contents of the grades array
private:
   string courseName; // course name for this grade book
   int grades[ students ][ tests ]; // two-dimensional array of grades
}; // end class GradeBook
```

Outline

fig07_23.cpp

(2 of 2)

```cpp
// Fig. 7.24: GradeBook.cpp
// Member-function definitions for class GradeBook that
// uses a two-dimensional array to store grades.
#include <iostream>
using std::cout;
using std::cin;
using std::endl;
using std::fixed;

#include <iomanip> // parameterized stream manipulators
using std::setprecision; // sets numeric output precision
using std::setw; // sets field width

// include definition of class GradeBook from GradeBook.h
#include "GradeBook.h"

// two-argument constructor initializes courseName and grades array
GradeBook::GradeBook( string name, const int gradesArray[][ tests ] )
{
   setCourseName( name ); // initialize courseName

   // copy grades from gradeArray to grades
   for ( int student = 0; student < students; student++ )

      for ( int test = 0; test < tests; test++ )
         grades[ student ][ test ] = gradesArray[ student ][ test ];
} // end two-argument GradeBook constructor

```

Outline

fig07_24.cpp

(1 of 7)

```
// function to set the course name
void GradeBook::setCourseName( string name )
{
   courseName = name; // store the course name
} // end function setCourseName

// function to retrieve the course name
string GradeBook::getCourseName()
{
   return courseName;
} // end function getCourseName

// display a welcome message to the GradeBook user
void GradeBook::displayMessage()
{
   // this statement calls getCourseName to get the
   // name of the course this GradeBook represents
   cout << "Welcome to the grade book for\n" << getCourseName() << "!"
      << endl;
} // end function displayMessage

// perform various operations on the data
void GradeBook::processGrades()
{
   // output grades array
   outputGrades();

   // call functions getMinimum and getMaximum
   cout << "\nLowest grade in the grade book is " << getMinimum()
      << "\nHighest grade in the grade book is " << getMaximum() << endl;
```

```
   // output grade distribution chart of all grades on all tests
   outputBarChart();
} // end function processGrades

// find minimum grade
int GradeBook::getMinimum()
{
   int lowGrade = 100; // assume lowest grade is 100

   // loop through rows of grades array
   for ( int student = 0; student < students; student++ )
   {
      // loop through columns of current row
      for ( int test = 0; test < tests; test++ )
      {
         // if current grade less than lowGrade, assign it to lowGrade
         if ( grades[ student ][ test ] < lowGrade )
            lowGrade = grades[ student ][ test ]; // new lowest grade
      } // end inner for
   } // end outer for

   return lowGrade; // return lowest grade
} // end function getMinimum

```

Outline

fig07_24.cpp

(3 of 7)

```
// find maximum grade
int GradeBook::getMaximum()
{
   int highGrade = 0; // assume highest grade is 0

   // loop through rows of grades array
   for ( int student = 0; student < students; student++ )
   {
      // loop through columns of current row
      for ( int test = 0; test < tests; test++ )
      {
         // if current grade greater than lowGrade, assign it to highGrade
         if ( grades[ student ][ test ] > highGrade )
            highGrade = grades[ student ][ test ]; // new highest grade
      } // end inner for
   } // end outer for

   return highGrade; // return highest grade
} // end function getMaximum

// determine average grade for particular set of grades
double GradeBook::getAverage( const int setOfGrades[], const int grades )
{
   int total = 0; // initialize total

   // sum grades in array
   for ( int grade = 0; grade < grades; grade++ )
      total += setOfGrades[ grade ];

```

```cpp
   // return average of grades
   return static_cast< double >( total ) / grades;
} // end function getAverage

// output bar chart displaying grade distribution
void GradeBook::outputBarChart()
{
   cout << "\nOverall grade distribution:" << endl;

   // stores frequency of grades in each range of 10 grades
   const int frequencySize = 11;
   int frequency[ frequencySize ] = { 0 };

   // for each grade, increment the appropriate frequency
   for ( int student = 0; student < students; student++ )

      for ( int test = 0; test < tests; test++ )
         ++frequency[ grades[ student ][ test ] / 10 ];

   // for each grade frequency, print bar in chart
   for ( int count = 0; count < frequencySize; count++ )
   {
      // output bar label ("0-9:", ..., "90-99:", "100:" )
      if ( count == 0 )
         cout << "  0-9: ";
      else if ( count == 10 )
         cout << "  100: ";
      else
         cout << count * 10 << "-" << ( count * 10 ) + 9 << ": ";

```


fig07_24.cpp

```
      // print bar of asterisks
      for ( int stars = 0; stars < frequency[ count ]; stars++ )
         cout << '*';

      cout << endl; // start a new line of output
   } // end outer for
} // end function outputBarChart

// output the contents of the grades array
void GradeBook::outputGrades()
{
   cout << "\nThe grades are:\n\n";
   cout << "            "; // align column heads

   // create a column heading for each of the tests
   for ( int test = 0; test < tests; test++ )
      cout << "Test " << test + 1 << "  ";

   cout << "Average" << endl; // student average column heading

   // create rows/columns of text representing array grades
   for ( int student = 0; student < students; student++ )
   {
      cout << "Student " << setw( 2 ) << student + 1;

```

Outline

fig07_24.cpp

(6 of 7)

```cpp
      // output student's grades
      for ( int test = 0; test < tests; test++ )
         cout << setw( 8 ) << grades[ student ][ test ];

      // call member function getAverage to calculate student's average;
      // pass row of grades and the value of tests as the arguments
      double average = getAverage( grades[ student ], tests );
      cout << setw( 9 ) << setprecision( 2 ) << fixed << average << endl;
   } // end outer for
} // end function outputGrades
```


fig07_24.cpp

```cpp
// Fig. 7.25: fig07_25.cpp
// Creates GradeBook object using a two-dimensional array of grades.

#include "GradeBook.h" // GradeBook class definition

// function main begins program execution
int main()
{
   // two-dimensional array of student grades
   int gradesArray[ GradeBook::students ][ GradeBook::tests ] =
      { { 87, 96, 70 },
        { 68, 87, 90 },
        { 94, 100, 90 },
        { 100, 81, 82 },
        { 83, 65, 85 },
        { 78, 87, 65 },
        { 85, 75, 83 },
        { 91, 94, 100 },
        { 76, 72, 84 },
        { 87, 93, 73 } };

   GradeBook myGradeBook(
      "CS101 Introduction to C++ Programming", gradesArray );
   myGradeBook.displayMessage();
   myGradeBook.processGrades();
   return 0; // indicates successful termination
} // end main
```


fig07_25.cpp

```
Welcome to the grade book for
CS101 Introduction to C++ Programming!

The grades are:

            Test 1  Test 2  Test 3  Average
Student  1      87      96      70    84.33
Student  2      68      87      90    81.67
Student  3      94     100      90    94.67
Student  4     100      81      82    87.67
Student  5      83      65      85    77.67
Student  6      78      87      65    76.67
Student  7      85      75      83    81.00
Student  8      91      94     100    95.00
Student  9      76      72      84    77.33
Student 10      87      93      73    84.33

Lowest grade in the grade book is 65
Highest grade in the grade book is 100

Overall grade distribution:
  0-9:
10-19:
20-29:
30-39:
40-49:
50-59:
60-69: ***
70-79: ******
80-89: ***********
90-99: *******
  100: ***
```


fig07_25.cpp

(2 of 2)

7.11 Introduction to C++ Standard Library Class Template `vector`

- **C-style pointer-based arrays**
  - **Have great potential for errors and several shortcomings**
    - **C++ does not check whether subscripts fall outside the range of the array.**
    - **Two arrays cannot be meaningfully compared with equality or relational operators.**
    - **One array cannot be assigned to another using the assignment operators.**

7.11 Introduction to C++ Standard Library Class Template `vector` (Cont.)

- **Class template `vector`**
  - **Available to anyone building applications with C++**
  - **Can be defined to store any data type.**
    - **Specified between angle brackets in `vector<` *type* `>`**
    - **All elements in a `vector` are set to `0` by default.**
  - **Member function `size` obtains size of array.**
    - **Number of elements as a value of type `size_t`**
  - **`vector` objects can be compared using equality and relational operators.**
  - **Assignment operator can be used for assigning `vector`s.**

7.11 Introduction to C++ Standard Library Class Template `vector` (Cont.)

- `vector` member function `at`
  - Provides access to individual elements.
  - Performs bounds checking.
    - Throws an exception when specified index is invalid
    - Accessing with square brackets does not perform bounds checking.

```cpp
// Fig. 7.26: fig07_26.cpp
// Demonstrating C++ Standard Library class template vector.
#include <iostream>
using std::cout;
using std::cin;
using std::endl;

#include <iomanip>
using std::setw;

#include <vector>
using std::vector;

void outputVector( const vector< int > & ); // display the vector
void inputVector( vector< int > & ); // input values into the vector

int main()
{
   vector< int > integers1( 7 ); // 7-element vector< int >
   vector< int > integers2( 10 ); // 10-element vector< int >

   // print integers1 size and contents
   cout << "Size of vector integers1 is " << integers1.size()
      << "\nvector after initialization:" << endl;
   outputVector( integers1 );

   // print integers2 size and contents
   cout << "\nSize of vector integers2 is " << integers2.size()
      << "\nvector after initialization:" << endl;
   outputVector( integers2 );
```

```cpp

   // input and print integers1 and integers2
   cout << "\nEnter 17 integers:" << endl;
   inputVector( integers1 );
   inputVector( integers2 );

   cout << "\nAfter input, the vectors contain:\n"
      << "integers1:" << endl;
   outputVector( integers1 );
   cout << "integers2:" << endl;
   outputVector( integers2 );

   // use inequality (!=) operator with vector objects
   cout << "\nEvaluating: integers1 != integers2" << endl;

   if ( integers1 != integers2 )
      cout << "integers1 and integers2 are not equal" << endl;

   // create vector integers3 using integers1 as an
   // initializer; print size and contents
   vector< int > integers3( integers1 ); // copy constructor

   cout << "\nSize of vector integers3 is " << integers3.size()
      << "\nvector after initialization:" << endl;
   outputVector( integers3 );

   // use overloaded assignment (=) operator
   cout << "\nAssigning integers2 to integers1:" << endl;
   integers1 = integers2; // integers1 is larger than integers2
```

Outline

fig07_26.cpp

(2 of 6)

```
   cout << "integers1:" << endl;
   outputVector( integers1 );
   cout << "integers2:" << endl;
   outputVector( integers2 );

   // use equality (==) operator with vector objects
   cout << "\nEvaluating: integers1 == integers2" << endl;

   if ( integers1 == integers2 )
      cout << "integers1 and integers2 are equal" << endl;

   // use square brackets to create rvalue
   cout << "\nintegers1[5] is " << integers1[ 5 ];

   // use square brackets to create lvalue
   cout << "\n\nAssigning 1000 to integers1[5]" << endl;
   integers1[ 5 ] = 1000;
   cout << "integers1:" << endl;
   outputVector( integers1 );

   // attempt to use out-of-range subscript
   cout << "\nAttempt to assign 1000 to integers1.at( 15 )" << endl;
   integers1.at( 15 ) = 1000; // ERROR: out of range
   return 0;
} // end main
```

```cpp

// output vector contents
void outputVector( const vector< int > &array )
{
   size_t i; // declare control variable

   for ( i = 0; i < array.size(); i++ )
   {
      cout << setw( 12 ) << array[ i ];

      if ( ( i + 1 ) % 4 == 0 ) // 4 numbers per row of output
         cout << endl;
   } // end for

   if ( i % 4 != 0 )
      cout << endl;
} // end function outputVector

// input vector contents
void inputVector( vector< int > &array )
{
   for ( size_t i = 0; i < array.size(); i++ )
      cin >> array[ i ];
} // end function inputVector
```


fig07_26.cpp

```
Size of vector integers1 is 7
vector after initialization:
           0           0           0           0
           0           0           0

Size of vector integers2 is 10
vector after initialization:
           0           0           0           0
           0           0           0           0
           0           0

Enter 17 integers:
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17

After input, the vectors contain:
integers1:
           1           2           3           4
           5           6           7
integers2:
           8           9          10          11
          12          13          14          15
          16          17

Evaluating: integers1 != integers2
integers1 and integers2 are not equal

Size of vector integers3 is 7
vector after initialization:
           1           2           3           4
           5           6           7
```

(continued at top of next slide)

fig07_26.cpp

(5 of 6)

(continued from bottom of previous slide)

```
Assigning integers2 to integers1:
integers1:
           8           9          10          11
          12          13          14          15
          16          17
integers2:
           8           9          10          11
          12          13          14          15
          16          17

Evaluating: integers1 == integers2
integers1 and integers2 are equal

integers1[5] is 13

Assigning 1000 to integers1[5]
integers1:
           8           9          10          11
          12        1000          14          15
          16          17

Attempt to assign 1000 to integers1.at( 15 )

abnormal program termination
```

Until Next Time

- **Review Chapter 7**
- **Preview Chapter 8**